

Practical 3 — A* Search on the 8-Puzzle

Run: `python P3_astar_8puzzle.py` | Library: `heapq` (priority queue ordered by $f = g + h$)

Real-life example used

This program solves the classic 3×3 sliding tile puzzle — the same physical toy sold as a small plastic square with 8 numbered tiles and one empty slot, where you slide tiles into the empty space to arrange the numbers in order. Here the puzzle starts scrambled and the program finds the shortest sequence of slides (Up / Down / Left / Right moves of the blank) that restores it to the solved arrangement, using the A* search algorithm with two different heuristics.

Program (P3_astar_8puzzle.py)

```
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def neighbors(state):
    i = state.index(0)
    r, c = divmod(i, 3)
    moves = []
    for dr, dc, name in [(-1,0,'Up'), (1,0,'Down'), (0,-1,'Left'), (0,1,'Right')]:
        nr, nc = r+dr, c+dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            ni = nr*3+nc
            new_state = list(state)
            new_state[i], new_state[ni] = new_state[ni], new_state[i]
            moves.append((tuple(new_state), name))
    return moves

def h_misplaced(state):
    return sum(1 for i in range(9) if state[i] != 0 and state[i] != GOAL[i])

def h_manhattan(state):
    dist = 0
    for i in range(9):
        val = state[i]
        if val == 0:
            continue
        goal_i = GOAL.index(val)
        r1, c1 = divmod(i, 3)
        r2, c2 = divmod(goal_i, 3)
        dist += abs(r1-r2) + abs(c1-c2)
    return dist

def astar(start, heuristic):
    counter = 0
    g_start = 0
    f_start = g_start + heuristic(start)
    open_heap = [(f_start, g_start, counter, start, [], None)]
    best_g = {start: 0}
    nodes_expanded = 0

    while open_heap:
        f, g, _, state, path, move = open_heap[0]
        f, g, _, state, path, move = heapq.heappop(open_heap)
        nodes_expanded += 1

        if state == GOAL:
            return path + [(state, move)], nodes_expanded

        if g > best_g.get(state, float('inf')):
            continue

        for new_state, move_name in neighbors(state):
```

```

        new_g = g + 1
        if new_g < best_g.get(new_state, float('inf')):
            best_g[new_state] = new_g
            new_f = new_g + heuristic(new_state)
            counter += 1
            heapq.heappush(open_heap, (new_f, new_g, counter, new_state,
                                      path + [(state, move)], move_name))

    return None, nodes_expanded

def print_board(state, label=""):
    print(label)
    for r in range(3):
        row = state[r*3:(r+1)*3]
        print(" ".join(str(x) if x != 0 else "_" for x in row))
    print()

if __name__ == "__main__":
    # Real-life scenario: a 3x3 sliding tile puzzle, like the classic
    # numbered sliding puzzle toy sold in stores. 0 represents the blank tile.
    start_state = (4, 1, 0, 2, 6, 3, 7, 5, 8)

    print("=" * 50)
    print("8-PUZZLE SOLVER USING A* SEARCH")
    print("=" * 50)
    print_board(start_state, "Start (Initial) Arrangement:")
    print_board(GOAL, "Goal Arrangement:")

    for name, heuristic in [("Misplaced Tiles", h_misplaced),
                           ("Manhattan Distance", h_manhattan)]:
        print("-" * 50)
        print(f"Heuristic: {name}")
        path, expanded = astar(start_state, heuristic)
        print(f"Nodes expanded: {expanded}")
        print(f"Number of moves to solve: {len(path) - 1}")
        move_sequence = [m for _, m in path if m is not None]
        print("Move sequence:", " -> ".join(move_sequence))
        print()

    print("=" * 50)
    print("STEP-BY-STEP SOLUTION (Manhattan Distance)")
    print("=" * 50)
    path, _ = astar(start_state, h_manhattan)
    for step, (state, move) in enumerate(path):
        label = "Start" if step == 0 else f"Step {step}: Move blank {move}"
        print_board(state, label)

```

Output

```
=====
8-PUZZLE SOLVER USING A* SEARCH
=====

Start (Initial) Arrangement:
4 1 _
2 6 3
7 5 8

Goal Arrangement:
1 2 3
4 5 6
7 8 _

-----
Heuristic: Misplaced Tiles
Nodes expanded: 14
Number of moves to solve: 8
Move sequence: Down -> Left -> Left -> Up -> Right -> Down -> Down -> Right

-----
Heuristic: Manhattan Distance
Nodes expanded: 11
Number of moves to solve: 8
Move sequence: Down -> Left -> Left -> Up -> Right -> Down -> Down -> Right

=====
STEP-BY-STEP SOLUTION (Manhattan Distance)
=====

Start
4 1 _
2 6 3
7 5 8

Step 1: Move blank Down
4 1 3
2 6 _
7 5 8

Step 2: Move blank Left
4 1 3
2 _ 6
7 5 8

Step 3: Move blank Left
4 1 3
_ 2 6
7 5 8

Step 4: Move blank Up
_ 1 3
4 2 6
7 5 8

Step 5: Move blank Right
1 _ 3
4 2 6
7 5 8

Step 6: Move blank Down
1 2 3
4 _ 6
7 5 8
```

Step 7: Move blank Down

1 2 3

4 5 6

7 _ 8

Step 8: Move blank Right

1 2 3

4 5 6

7 8 _